



VERIQTA

“ Master the foundation. Build with confidence. ”

KUBERNETES FUNDAMENTALS

VOLUME 1

From Containers to Your First Production Cluster

Understand the building blocks



Deploy with clarity. Operate like a pro.



CONTAINERS



PODS



SERVICES



STORAGE



SECURITY

20 PAGES OF HANDWRITTEN NOTES



BEGINNER FRIENDLY



DIAGRAMS & EXPLANATIONS



REAL-WORLD EXAMPLES



CHECKLISTS & CHEAT SHEETS



PRODUCTION READY

★ STRONG FOUNDATION. LIMITLESS POSSIBILITIES. ★



1. INTRODUCTION TO KUBERNETES

“Automate everything, Orchestrate anything.”

① WHAT IS KUBERNETES?

Kubernetes (K8s) is an open-source container orchestration platform used to automate deployment, scaling, and management of containerized applications.



Kubernetes doesn't run your containers, it runs your containerized applications at scale.

② WHY KUBERNETES EXISTS?

- ❑ Managing containers with Docker alone is complex.
- ❑ Scaling, self-healing, and rolling updates are hard to do manually.
- ❑ Applications grow. Infrastructure changes. Failures happen.
- ❑ Kubernetes automates the hard parts so you can focus on building.

Problem with Docker Alone

- ✗ No auto-scaling
- ✗ No self-healing
- ✗ Manual rollouts
- ✗ Hard to manage multiple containers
- ✗ No service discovery
- ✗ No resource management
- ✗ Not built for production

③ CONTAINER ORCHESTRATION

Container orchestration is the automated management of containers throughout their lifecycle.

Kubernetes handles:



④ KUBERNETES FEATURES

- ✓ Automated Scheduling - Places containers on the best node
- ✓ Self-Healing - Restarts, replaces or reschedules failed containers
- ✓ Horizontal Scaling - Scale apps up or down easily
- ✓ Rolling Updates & Rollbacks - Update without downtime
- ✓ Service Discovery & Load Balancing - Built-in networking
- ✓ Storage Orchestration - Mount and manage storage
- ✓ Secret & Config Management - Keep sensitive data safe
- ✓ Extensible - Plug in your tools (CNI, CSI, CRI, etc.)
- ✓ Declarative API - Define desired state, K8s makes it real

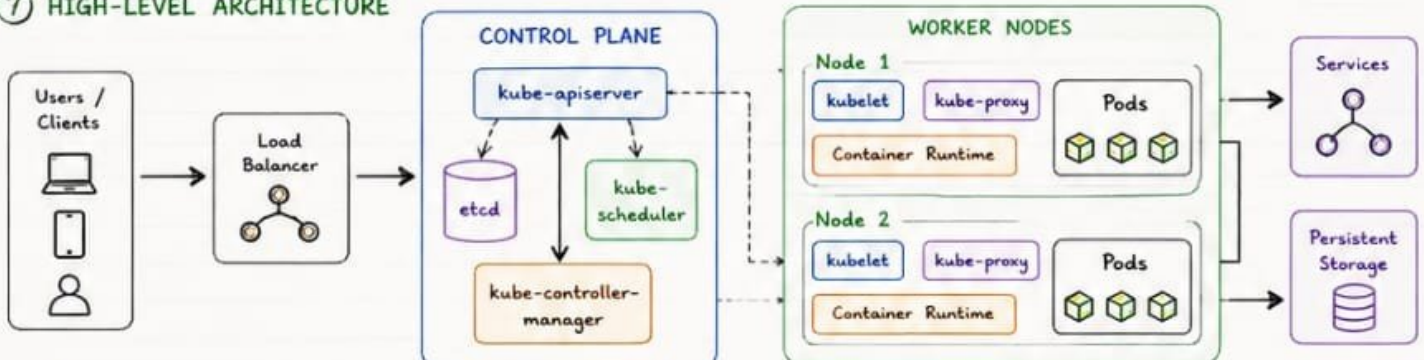
⑤ REAL-WORLD USE CASES

- Microservices → Run and scale many services
- E-commerce Platforms → Handle high traffic & spikes
- AI / ML Workloads → Train, deploy, and scale models
- Enterprise Apps → Run critical business applications
- Hybrid / Multi-Cloud → Run anywhere, move anywhere
- DevOps & CI/CD → Automate delivery at scale

⑥ BENEFITS

- ★ High Availability - Keeps your apps running
- ★ Scalability - Scale to any size easily
- ★ Portability - Run on any cloud or on-prem
- ★ Efficiency - Better resource utilization
- ★ Resilience - Auto recover from failures
- ★ Developer Productivity - Focus on code, not infra
- ★ Cost Optimization - Use resources smartly

⑦ HIGH-LEVEL ARCHITECTURE



Kubernetes is the operating system for containers.

It takes care of the infrastructure so you can focus on your application.

Key Takeaway

Kubernetes automates deployment, scaling, and management of containerized applications.



2. WHY KUBERNETES WAS CREATED

“ Docker is great for running containers, but not for running a containerized platform. ”

① THE LIMITATIONS OF DOCKER ALONE

Docker solves packaging and runtime, but not the entire lifecycle of applications at scale.

- ✗ No built-in orchestration
- ✗ No auto-scaling
- ✗ No self-healing (restart, replace)
- ✗ No rolling updates / rollbacks
- ✗ No service discovery or load balancing
- ✗ No centralized configuration or secrets
- ✗ No storage orchestration
- ✗ No scheduling (which container goes where?)
- ✗ Manual operations don't scale



② SCALING CONTAINERS IS HARD

As your application grows, you must handle:



Scale up
or down



Discover
services



Balance
traffic



Self-heal
failures



Roll out
updates



Use resources
efficiently



Manage
storage



Manage
secrets & configs



Automate
operations

Manually doing all this is error-prone, slow and not scalable!

We needed an orchestration platform.

③ GOOGLE BORG - THE INSPIRATION

Google



- Borg is Google's internal system used to run billions of containers across thousands of machines.
- Borg handled scheduling, scaling, load balancing, failures, and more - automatically.
- Borg proved that large-scale container orchestration was possible.

④ THE BIRTH OF KUBERNETES



Kubernetes

- In 2014, Google open-sourced Borg-like technology as "Kubernetes" (K8s).
- Built by Google engineers based on over a decade of running Borg.
- Goal: Provide a portable, extensible, open-source orchestration platform for everyone.

⑤ CNCF - CLOUD NATIVE COMPUTING FOUNDATION



CNCF

- Kubernetes was one of the first projects hosted by the Cloud Native Computing Foundation (CNCF)
- CNCF promotes open source, vendor neutral cloud native technologies.
- Today, Kubernetes is one of the most successful open-source projects in the world.

CNCF PROJECTS INCLUDE:

- Kubernetes
- Prometheus
- Envoy
- CoreDNS
- Istio
- etc.



⑥ KUBERNETES EVOLUTION TIMELINE



KEY TAKEAWAY

Docker helps you build containers.
Kubernetes helps you run containers at scale reliably.



Docker

Build, Ship
& Run Containers

+



Kubernetes

Orchestrate, Scale
& Manage Containers

=



Production-Ready
Applications
at Any Scale!

REMEMBER



Kubernetes takes care of the complexity so you can focus on building great applications.



3. KUBERNETES ARCHITECTURE OVERVIEW

“Kubernetes follows a Master-Worker architecture.”

① CLUSTER ARCHITECTURE

A Kubernetes cluster consists of:

- **Control Plane** - Brain of the cluster
- **Worker Nodes** - Where your applications run
- **Add-ons** - DNS, Ingress, Monitoring, Storage, etc.

★ Everything in Kubernetes is an API object. The API Server is the **single entry point** for all operations.

② CONTROL PLANE (THE BRAIN)

Manages the cluster and makes global decisions about:

- Scheduling
- Detecting & responding to failures
- Managing workloads
- Maintaining desired state



The Control Plane **never** runs your application containers. It only **manages** the cluster.

③ WORKER NODES (THE MUSCLES)

Worker nodes run your application containers and handle:

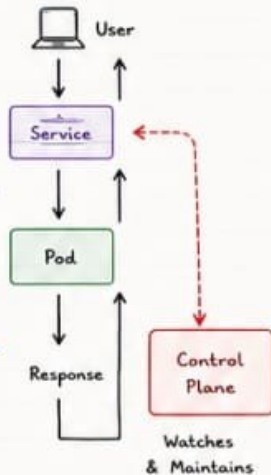
- Running **Pods**
- Networking
- Storage
- Health checks via kubelet



Worker nodes take instructions from the Control Plane and make it happen.

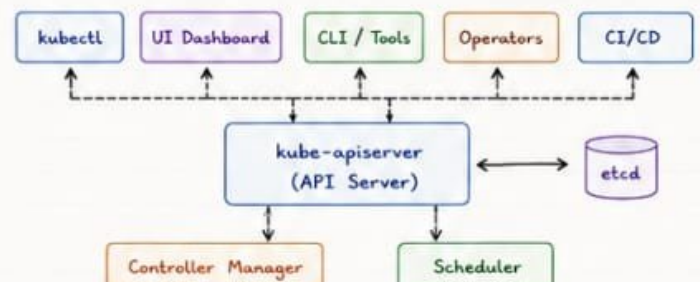
④ REQUEST FLOW (HIGH LEVEL)

- 1 User sends request to the application (via Service / LoadBalancer / Ingress)
- 2 Request reaches the Service (kube-proxy handles routing)
- 3 Traffic is forwarded to one of the Pods (running on Worker Node)
- 4 Pod processes the request and returns the response
- 5 Response goes back through the Service to the user
- 6 Control Plane continuously watches and ensures desired state

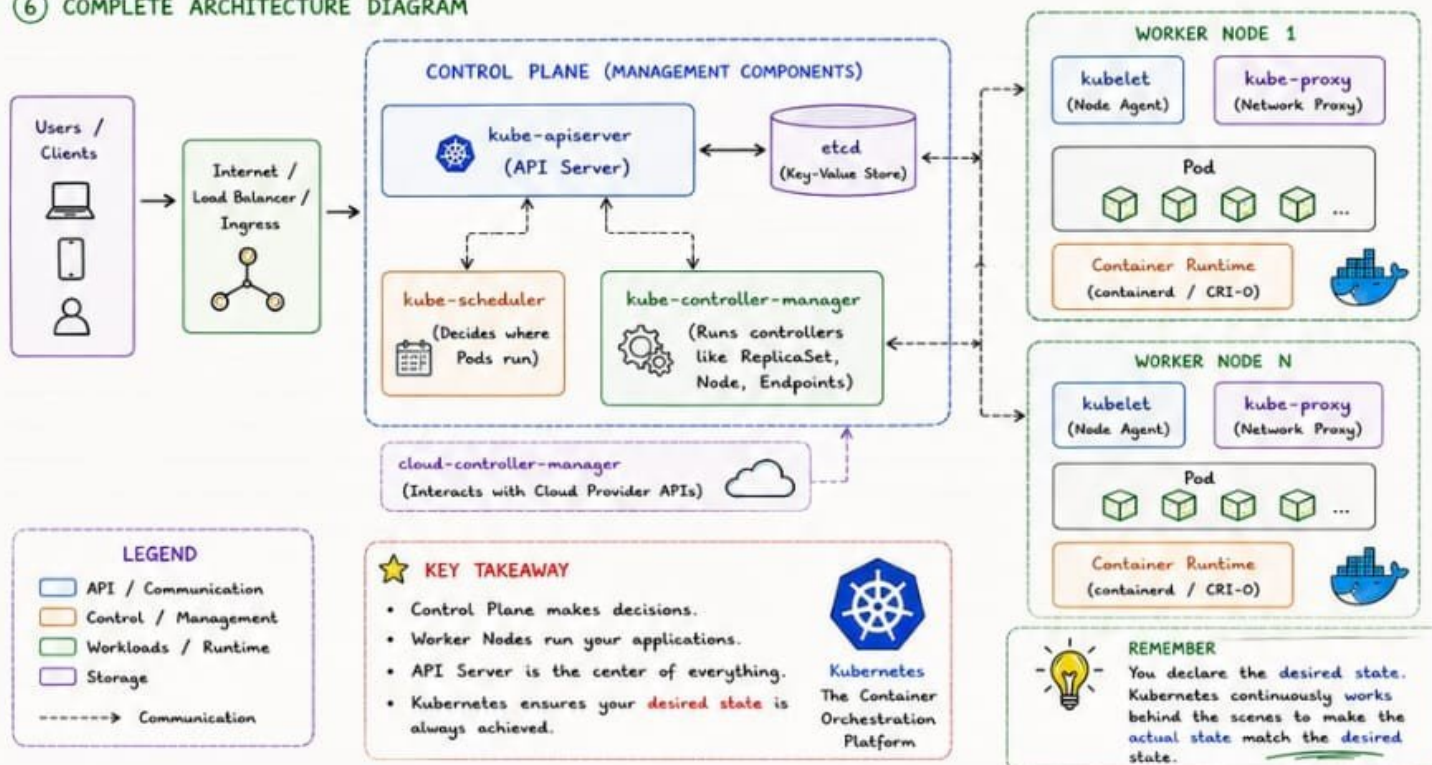


⑤ API-FIRST DESIGN

- ✓ Everything in Kubernetes is accessed through the API Server.
- ✓ **kubectl**, UI, CLI, operators, controllers, and even other services - all talk to the API Server.
- ✓ The API Server validates, authenticates, and manages all cluster interactions.
- ✓ This makes Kubernetes extensible, automatable, and platform-agnostic.



⑥ COMPLETE ARCHITECTURE DIAGRAM



LEGEND

- API / Communication
- Control / Management
- Workloads / Runtime
- Storage
- > Communication

★ KEY TAKEAWAY

- Control Plane makes decisions.
- Worker Nodes run your applications.
- API Server is the center of everything.
- Kubernetes ensures your **desired state** is always achieved.



Kubernetes
The Container
Orchestration
Platform



REMEMBER

You declare the desired state. Kubernetes continuously works behind the scenes to make the **actual state** match the desired state.



4. CONTROL PLANE COMPONENTS

“ The Brain of the Kubernetes Cluster. ”

The Control Plane manages the cluster and makes **global decisions** about the cluster (scheduling, health, scaling, updates, etc).

① KUBE-APISERVER

API
SERVER

- The front-end of the control plane.
- Exposes the Kubernetes API.
- All requests (from users, kubectl, controllers, nodes) go through it.
- Validates, authenticates, and authorizes requests.
- Talks to all other control plane components and etcd.

★ Think: The API Server is the “door” to the cluster.

② ETCD

ETCD
(KEY-VALUE STORE)

- Consistent and highly-available key-value database.
- Stores all cluster data and state (the **source of truth**).
- Data is written to etcd through the API Server.
- If etcd is down, the cluster cannot work.

★ Think: etcd is the cluster's **memory**.

③ KUBE-SCHEDULER



SCHEDULER

- Watches for newly created **Pods** that have no node assigned.
- Decides which node is the best fit for the Pod.
- Considers resources, constraints, affinity/anti-affinity, taints/tolerations, and policies.
- Binds the Pod to a Node via the API Server.

★ Think: The scheduler is the “matchmaker” between Pods and Nodes.

④ KUBE-CONTROLLER-MANAGER

CONTROLLER
MANAGER

- Runs controller loops that constantly watch the cluster state.
- Ensures the actual state matches the desired state.
- Examples: Node Controller, Replication Controller, Endpoint Controller, Job Controller, Service Account Controller, etc.

★ Think: The controller manager is the “caretaker” of the **desired state**.

⑤ CLOUD-CONTROLLER-MANAGER

CLOUD
CONTROLLER
MANAGER

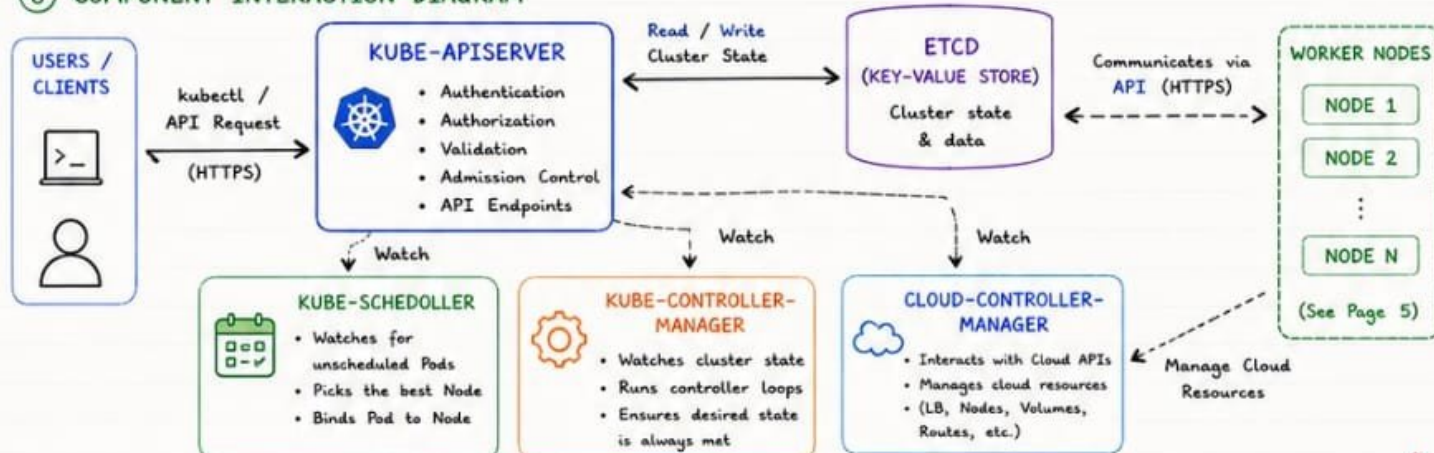
- Interacts with cloud provider APIs.
- Handles cloud-specific tasks:
 - Provisioning Load Balancers
 - Managing Node (instances)
 - Attaching Volumes
 - Managing Routes, etc.

★ Think: The cloud controller is the “bridge” between Kubernetes and your cloud.

CONTROL PLANE SUMMARY

- ★ **API Server** - Handles all requests and communication.
- ★ **etcd** - Stores all cluster data.
- ★ **Scheduler** - Decides where Pods run.
- ★ **Controller Manager** - Keeps the cluster in the desired state.
- ★ **Cloud Controller Manager** - Talks to the cloud provider.

⑥ COMPONENT INTERACTION DIAGRAM



★ KEY TAKEAWAY



The Control Plane components work together as a team:
The API Server is the **hub**, etcd is the **memory**, the Scheduler **picks** where to run, the Controller Manager keeps everything **healthy**, and the Cloud Controller Manager handles the **outside world**.

REMEMBER

- ✓ No API Server → No cluster
- ✓ No etcd → No data / No cluster
- ✓ No Scheduler → Pods won't be placed
- ✓ No Controller Manager → State drifts
- ✓ No Cloud Controller → Cloud features break



5. WORKER NODE COMPONENTS

“Worker Nodes **run** your containers and keep them **healthy**.”

① KUBELET



KUBELET

- Primary “agent” that runs on each worker node.
- Registers the node with the API Server.
- Watches for PodSpecs assigned to this node.
- Ensures containers are running as expected (health via probes).
- Reports node and Pod status back to the Control Plane.

★ Think: kubelet is the **hands-on worker** that makes things happen on the node.

② KUBE-PROXY



KUBE-PROXY

- Network proxy that runs on every node.
- Maintains network rules (iptables / IPVS) to enable Service networking.
- Ensures network traffic reaches the correct Pods.
- Handles load balancing for Services.

★ Think: kube-proxy is the **traffic cop** for Services.

③ CONTAINER RUNTIME



CONTAINER RUNTIME

- Responsible for pulling container images, creating containers, and managing them.
- Popular runtimes: containerd, CRI-O.
- Implements the Container Runtime Interface (CRI) used by kubelet.
- Handles low-level container operations.

★ Think: Runtime is the **engine** that runs the containers.

④ PODS

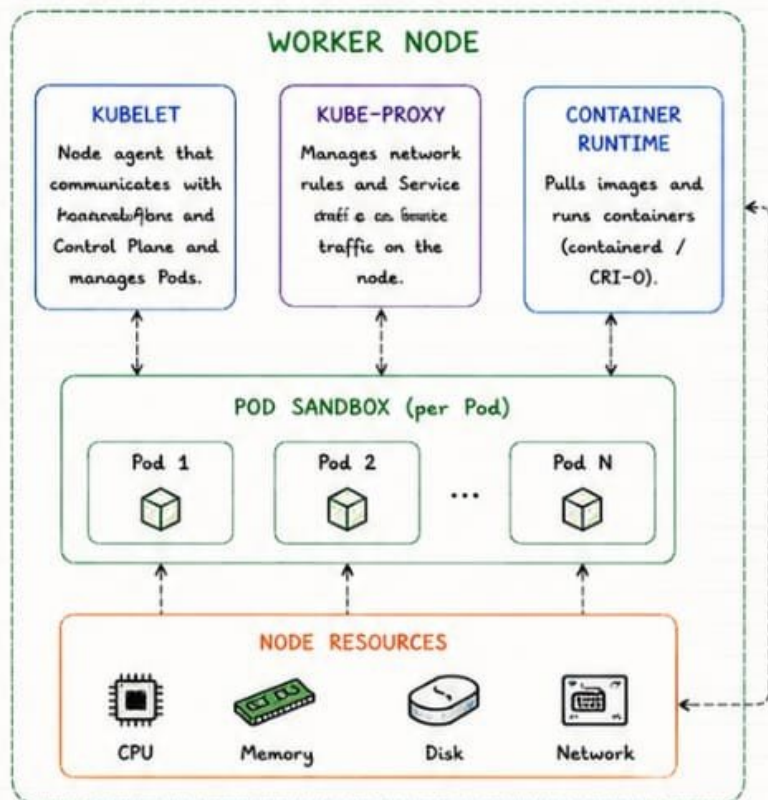


POD

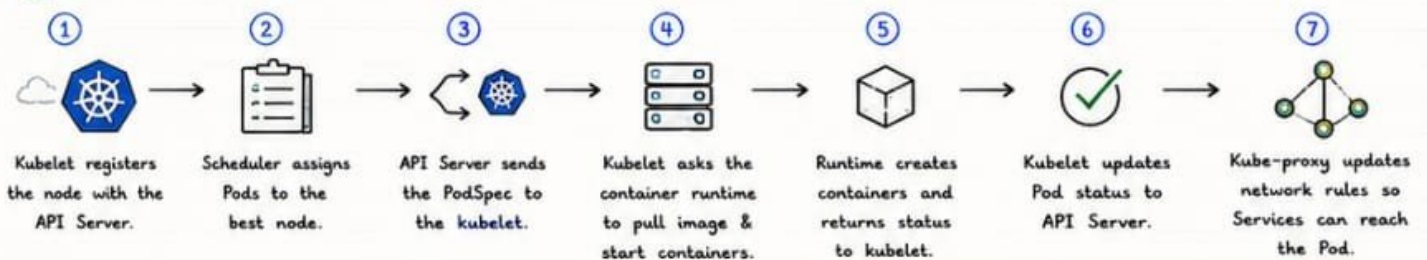
- Smallest deployable unit in Kubernetes.
- One or more containers that share the same network and storage.
- Scheduled to run on a worker node.
- Pods are ephemeral. Controllers keep them running.

★ Think: A Pod is like a **box** that holds one or more containers together.

⑤ WORKER NODE ARCHITECTURE



⑥ NODE WORKFLOW (HOW IT WORKS)



★ KEY TAKEAWAY

Worker Nodes are where your applications live.
Kubelet is the **manager**, kube-proxy handles networking,
and the container runtime runs your containers.



QUICK RECAP

- ✓ Kubelet → Manages Pods on the node
- ✓ Kube-proxy → Manages network traffic
- ✓ Container Runtime → Runs containers
- ✓ Pods → Your workloads
- ✓ Together → Make the node productive!



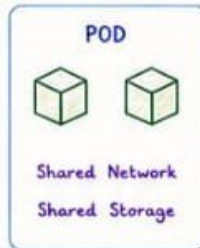


6. PODS

“ A Pod is the smallest and simplest deployable unit in Kubernetes. ”

① WHAT IS A POD?

- A Pod represents one or more containers that share the same network, storage, and specifications.
- Pods are ephemeral (temporary).
- Pods run on a Node.
- Kubernetes manages Pods, not containers directly.



★ Think: A Pod is like an envelope that holds one or more containers together.

② POD LIFECYCLE



Pod is accepted by the cluster but not yet scheduled.

Pod is bound to a node and containers are running.

All containers completed successfully.

At least one container failed and will not restart.

The state of the Pod cannot be determined.



Pods are disposable. When a Pod dies, Kubernetes creates a new one to match the desired state.

③ SINGLE-CONTAINER PODS

```

apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  containers:
  - name: nginx
    image: nginx:latest
    ports:
    - containerPort: 80
  
```

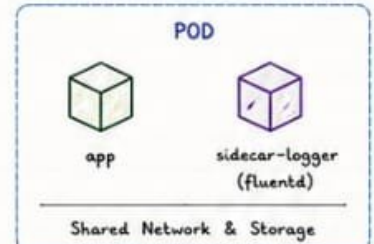


- Most common type.
- One container does one job.
- Ideal for stateless applications.

④ MULTI-CONTAINER PODS

```

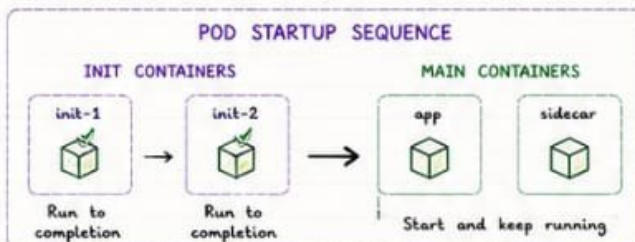
apiVersion: v1
kind: Pod
metadata:
  name: app-pod
spec:
  containers:
  - name: app
    image: myapp:1.0
  - name: sidecar-logger
    image: fluentd:latest
  
```



- Containers in the same Pod share the same network namespace and storage volumes.
- Use cases: sidecars, adapters, log collectors, proxy, etc.

⑤ INIT CONTAINERS

- Init containers run before the main containers.
- They complete their task and exit.
- Main containers start only after all init containers succeed.
- Common for setup, migrations, permissions, configs, etc.



★ Init containers are perfect for preparing the environment before the main application starts.

⑥ PRODUCTION BEST PRACTICES

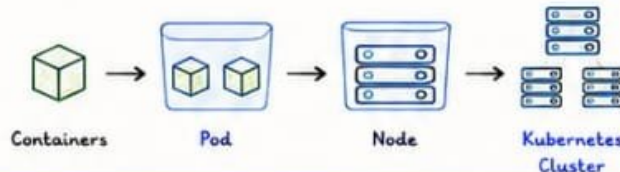
- ✓ Keep Pods small and focused (one main purpose).
- ✓ One Pod = One application (avoid packing unrelated containers together).
- ✓ Use resource requests and limits.
- ✓ Use liveness, readiness, and startup probes.
- ✓ Do not store critical data in Pods (use Persistent Volumes).
- ✓ Design Pods to be disposable and replaceable.
- ✓ Use labels and annotations for organization.
- ✓ Use anti-affinity / affinity for better scheduling.
- ✓ Monitor and log your Pods.

POD IS NOT A...

- ✗ Long-term storage
- ✗ Directly exposed to the internet
- ✗ Replacement for a VM
- ✗ A place to run unrelated containers

★ KEY TAKEAWAY

Pods are the building blocks of Kubernetes. They hold your containers, run on nodes, and give you the power to build scalable, resilient applications.



REMEMBER



Small, focused, disposable. That's the Kubernetes way! Pods come and go, your application keeps running.



7. REPLICASETS

“ReplicaSet ensures the **desired** number of Pod replicas are running at any time.”

① DESIRED STATE

- ReplicaSet is a controller that ensures a specified number of Pod replicas are running at all times.
- You declare the desired state (number of replicas), and the ReplicaSet works continuously to achieve and maintain it.
- It does not create Pods directly (in most cases) - it creates them through its controller logic.

DESIRED STATE

replicas: 3

Always 3 Pods
Running

☆ Think: Desired State = What you want.
ReplicaSet = What makes it happen.

② REPLICA MANAGEMENT

- Monitors the number of Pods matching its selector.
- Creates new Pods if the current count is less than the desired count.
- Deletes extra Pods if the current count is more than the desired count.
- Uses the Pod template to create new Pods.

replicas: 3

CURRENT		ACTION
0	→	Create 3
1	→	Create 2
2	→	Create 1
3	→	Do nothing
4	→	Delete 1
5+	→	Delete extra



ReplicaSet is always working in the background to match the **desired state**.

③ SELF-HEALING

- If a Pod crashes, gets deleted, or a node fails, the ReplicaSet immediately creates a new Pod.
- This automatic healing ensures application availability and reliability.
- It is the core reason why ReplicaSets are the building block of Deployments.

SELF-HEALING EXAMPLE



☆ ReplicaSet keeps your app alive!

④ SCALING

- You can scale a ReplicaSet up or down manually or with automation.
- Scaling changes only the desired replica count.
- Horizontal scaling = more replicas (handles more load).
- Vertical scaling (resources) is done in the Pod template.

SCALE EXAMPLES

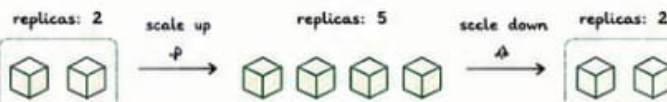
Scale up to 5

```
kubectl scale rs my-rs
--replicas=5
```

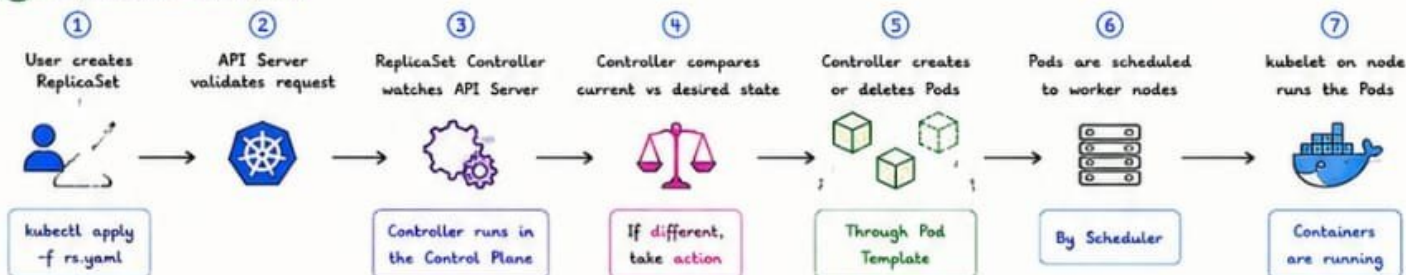
Scale down to 2

```
kubectl scale rs my-rs
--replicas=2
```

SCALING FLOW



⑤ REPLICASET WORKFLOW



⑥ PRODUCTION EXAMPLES

USE CASE	HOW REPLICASET HELPS
Web Applications	Ensures the right number of web server Pods are always running.
Microservices	Keeps microservice instances alive and scalable.
Background Workers	Maintains worker Pods that process jobs or messages.
APIs & Backend	Handles spikes in traffic by scaling replicas.
E-commerce	Keeps critical services (cart, checkout, payment) highly available.

★ KEY TAKEAWAY

- ReplicaSet ensures the desired number of Pod replicas.
- It provides self-healing and high availability.
- It is used directly or (more commonly) by Deployments.



REMEMBER

- ✓ ReplicaSet ≠ Creates apps by itself (use Deployment)
- ✓ Uses selector to manage Pods
- ✓ Always works to match the desired state
- ✓ Core building block for Kubernetes applications



QUICK COMMANDS

```
# Get ReplicaSets
kubectl get rs

# Describe ReplicaSet
kubectl describe rs <name>

# Scale ReplicaSet
kubectl scale rs <name> --replicas=<n>

# Delete ReplicaSet
kubectl delete rs <name>
```

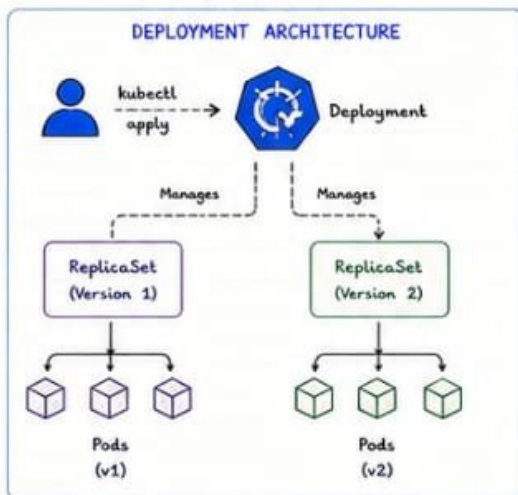



8. DEPLOYMENTS

“Deployments manage ReplicaSets and provide declarative updates for Pods.”

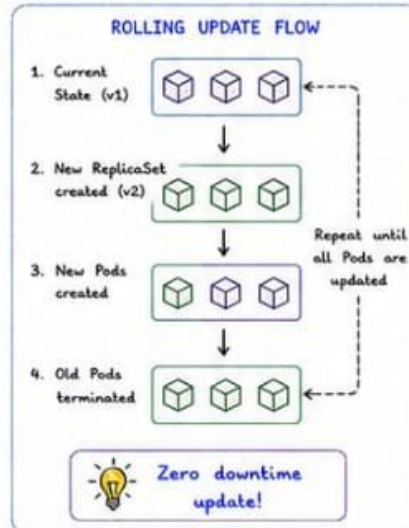
① DEPLOYMENT ARCHITECTURE

- A Deployment provides declarative updates for Pods and ReplicaSets.
- You define the desired state in a Deployment, and Kubernetes changes the actual state to the desired state at a controlled rate.
- It creates and manages ReplicaSets.



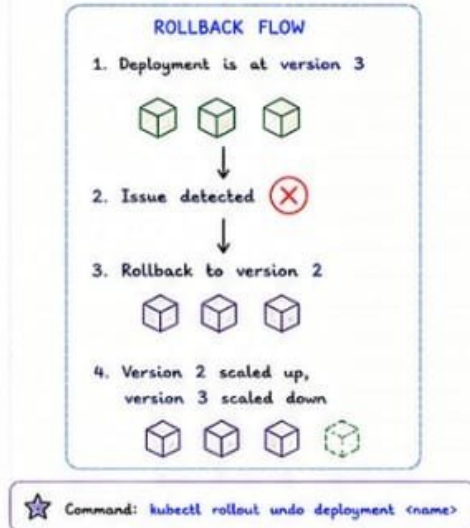
② ROLLING UPDATES

- Updates are applied gradually so your application stays available.
- New ReplicaSet is created.
- Pods are updated one by one.
- Old Pods are terminated after new Pods are ready.



③ ROLLBACKS

- If something goes wrong, you can roll back to a previous ReplicaSet.
- Kubernetes brings back the old ReplicaSet and scales it up.
- Fast and safe recovery.



④ REVISION HISTORY

- Every change creates a new revision.
- Kubernetes keeps track of the history.
- You can view history and rollback to any revision.

REVISION HISTORY EXAMPLE

REVISION	CHANGE CAUSE	IMAGE	CREATED AT	STATUS
1	Initial deployment	nginx:1.19	May 10, 10:00	Active
2	Update image	nginx:1.20	May 11, 09:30	Superseded
3	Update resources	nginx:1.21	May 12, 14:15	Active
4	Bad update	nginx:1.22	May 13, 08:45	Failed
5	Rollback to rev 3	nginx:1.21	May 13, 09:00	Active

View history

`kubectl rollout history deployment <name>`

Rollback to a specific revision

`kubectl rollout undo deployment <name> --to-revision=3`

⑤ DEPLOYMENT STRATEGIES

A. RECREATE STRATEGY

- Terminates all old Pods before creating new ones.
- Downtime occurs.
- Simple but not suitable for production.

Old Pods Terminated



New Pods Created



B. ROLLING UPDATE STRATEGY (Default)

- Gradually replaces old Pods with new Pods.
- No downtime.
- Recommended for most applications.

Gradual Replacement



★ Strategy is defined in the Deployment spec under: `.spec.strategy`
Type: `RollingUpdate` (default) or `Recreate`

⑥ BEST PRACTICES

- ✓ Use RollingUpdate strategy (default).
- ✓ Always set resource requests and limits.
- ✓ Use liveness and readiness probes.
- ✓ Keep your images small and secure.
- ✓ Use meaningful labels and annotations.
- ✓ Test updates in staging before production.
- ✓ Monitor your deployments and set alerts.
- ✓ Use `kubectl rollout status` to monitor updates.
- ✓ Avoid unnecessary updates.
- ✓ Document your changes and maintain revision history.

SIMPLE DEPLOYMENT EXAMPLE

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
      maxSurge: 1
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.21
          ports:
            - containerPort: 80
```

⑦ DEPLOYMENT WORKFLOW



★ KEY TAKEAWAY



Deployments make updates safe, controlled, and reliable. They ensure high availability and make rollbacks simple.

REMEMBER

- ✓ Define desired state
- ✓ Kubernetes does the rest
- ✓ Updates without downtime
- ✓ Rollback when needed



“Deployments give you control, stability, and confidence in production.”



9. NAMESPACES

“Namespaces provide a way to **divide** cluster resources between multiple users, teams, or projects.”

① DEFAULT NAMESPACE

- When you create resources without specifying a namespace, they are created in the **default** namespace.
- It exists by default in every Kubernetes cluster.
- Not recommended for production workloads.

DEFAULT NAMESPACE

default

(Default Namespace)



Resources created without specifying a namespace go here.

★ Think: default is for **testing** only, not for production.

③ KUBE-PUBLIC NAMESPACE

- This namespace is public and readable by all users (even unauthenticated users).
- Primarily used for cluster-wide resources that should be visible to everyone.
- Rarely used in most setups.

kube-public

(Public Namespace)



Cluster information that can be read by all users (including unauthenticated).

★ Think: Public = **Readable**, not necessarily safe for your apps.

⑤ MULTI-TENANCY

- Namespaces enable multiple teams or tenants to share the same cluster.
- Each team gets their own namespace with isolated resources and quotas.
- Ideal for organizations, SaaS platforms, and managed Kubernetes services.

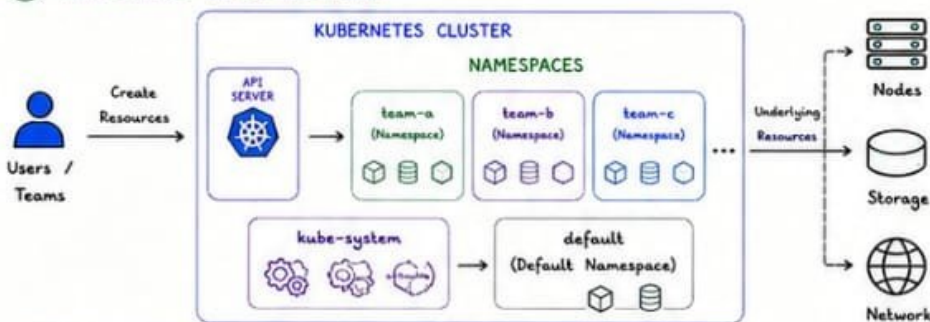
MULTI-TENANCY EXAMPLE



One Cluster, Multiple Tenants
Isolated, Secure, Scalable

★ Think: Many teams, one cluster, **zero** interference.

⑥ NAMESPACE ARCHITECTURE



Namespaces provide logical separation. The cluster provides physical resources.

★ KEY TAKEAWAY

- Namespaces help you **ORGANIZE**, **ISOLATE**, and **SECURE** resources.
- Use meaningful names for your projects and teams.
- Keep system namespaces untouched.
- Plan your namespace strategy early in your Kubernetes journey.



② KUBE-SYSTEM NAMESPACE

- Used by Kubernetes to run system components.
- Critical add-ons and controllers live here.
- You should **NOT** deploy your applications in this namespace.

kube-system

(System Namespace)



- CoreDNS
- kube-proxy
- kube-controller-manager
- Metrics Server
- etc.

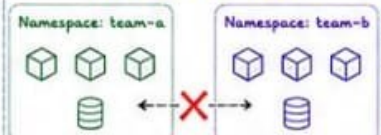


Think: **Hands off!** Let Kubernetes manage this.

④ RESOURCE ISOLATION

- Namespaces isolate resources like Pods, Services, Deployments, ConfigMaps, etc.
- Resources in one namespace cannot see others by default.
- Helps avoid conflicts and improves security.

ISOLATION EXAMPLE



team-a cannot access resources in team-b (by default)



Think: Namespaces = **Boundaries** between teams/apps.

⑦ COMMON NAMESPACES

NAMESPACE	PURPOSE
default	Default namespace for resources (use for testing only)
kube-system	Kubernetes system components and add-ons
kube-public	Publicly readable resources
kube-node-lease	Node heartbeat information (leases)
Custom	Created for teams, projects, or applications

REMEMBER

- default → Not for production
- kube-system → System components
- kube-public → Publicly readable
- Use namespaces for isolation and multi-tenancy
- Namespaces ≠ Security boundary (use RBAC too!)



10. LABELS & SELECTORS

“Labels organize your resources. Selectors find them.”

① LABELS

- Labels are key-value pairs attached to Kubernetes objects.
- Used to organize, identify and group resources.
- Labels do not provide uniqueness.
- Best practice: use consistent, meaningful labels.

★ Think: Labels are for humans.
Selectors are for controllers.

EXAMPLE - POD WITH LABELS

```
yaml
apiVersion: v1
kind: Pod
metadata:
  name: web-pod
  labels:
    app: web
    tier: frontend
    env: production
    version: v1
```



② MATCHLABELS

- The simplest way to select resources.
- All specified labels must match exactly.
- Supports equality-based matching only.

MATCHLABELS EXAMPLE

```
yaml
selector:
  matchLabels:
    app: web
    tier: frontend

# Selects resources where
# app=web AND tier=frontend
```



Think: matchLabels = AND (all must match)

③ MATCHEXPRESSIONS

- More advanced and flexible.
- Allows set-based requirements.
- Supports operators: In, NotIn, Exists, DoesNotExist.
- Multiple expressions are ANDed together.

MATCHEXPRESSIONS EXAMPLE

```
yaml
selector:
  matchExpressions:
    - key: env
      operator: In
      values: ["prod", "staging"]
    - key: tier
      operator: NotIn
      values: ["backend"]

# env is prod OR staging
# AND tier is not backend
```

★ Think: matchExpressions = Powerful filtering.
Use when matchLabels is not enough.

④ SELECTORS

- Selectors are used by controllers (ReplicaSet, Deployment, Service, etc.) to find matching resources.
- If a resource's labels match the selector, it becomes part of that resource set.
- Changing labels can add or remove resources from the selection.

WHERE SELECTORS ARE USED

- ReplicaSet / Deployment
- Service
- Job / CronJob
- NetworkPolicy
- PodDisruptionBudget

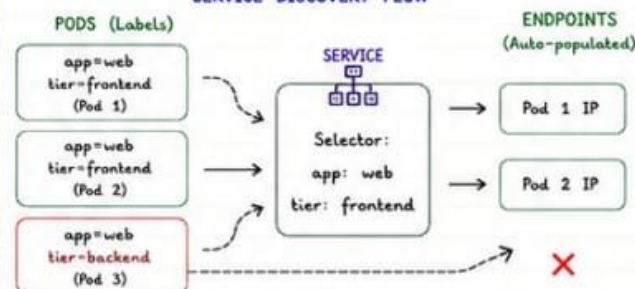


Think: Controllers watch selectors and act on matches.
Labels change → membership changes.

⑤ SERVICE DISCOVERY

- Services use selectors to automatically discover Pods.
- No need to update Endpoints manually.
- When Pods with matching labels are created or deleted, the Service updates automatically.

SERVICE DISCOVERY FLOW



★ Think: Service doesn't know Pods by IP.
It finds them using Labels + Selectors.

SERVICE EXAMPLE

```
yaml
apiVersion: v1
kind: Service
metadata:
  name: web-svc
spec:
  selector:
    app: web
    tier: frontend
  ports:
    - port: 80
      targetPort: 8080
```

⑥ PRODUCTION LABELING STRATEGY

RECOMMENDED LABELS

LABEL KEY	EXAMPLE	PURPOSE
app.kubernetes.io/name	payment-api	Application name
app.kubernetes.io/instance	payment-api-prod	Instance/Release name
app.kubernetes.io/component	api	Component (api, web, worker)
app.kubernetes.io/part-of	payment-platform	Part of larger application
app.kubernetes.io/managed-by	helm	Deployed by (helm, kustomize, etc.)
environment	production	Environment (dev, staging, prod)
tier	frontend	Tier (frontend, backend, database)
version	v1.2.3	Application version

BEST PRACTICES

- ✓ Use consistent label keys.
- ✓ Follow Kubernetes recommended labels for portability.
- ✓ Keep labels immutable (don't change frequently).
- ✓ Use labels for selection, not identity.
- ✓ Document your labeling standard.
- ✓ Avoid long or sensitive values.

BAD PRACTICES

- ✗ Using labels as unique IDs.
- ✗ Changing labels often.
- ✗ Using unstructured keys (e.g., "name", "type").
- ✗ Storing business data in labels.



★ KEY TAKEAWAY

Labels describe your resources. Selectors find them. Together, they power automation, scaling, and discovery.





11. SERVICES

“ Services provide a **stable network endpoint** to access a set of Pods. ”

① WHAT IS A SERVICE?

- A Service is an abstraction that defines a logical set of Pods and a policy to access them.
- Pods are ephemeral (change IP).
- Services provide a stable IP and DNS name.
- Built-in load balancing.
- Enables decoupling of frontend and backend.

SERVICE BENEFITS

- ✓ Stable endpoint
- ✓ Load balancing
- ✓ Service discovery
- ✓ Decoupling
- ✓ Scalability

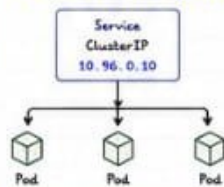
★ Think: Pods come and go,
Services **stay and route**.

② SERVICE TYPES OVERVIEW

TYPE	EXPOSED TO	USE CASE	PORTS
ClusterIP	Inside the cluster	Internal communication	Cluster IP (Virtual IP)
NodePort	All nodes' IP:NodePort	Access from outside (via node IP)	NodePort (30000-32767)
LoadBalancer	External (via Cloud LB)	Production apps (cloud environments)	Port / LB (e.g. 80/443)
ExternalName	Maps to external DNS name	Access external services	No proxy (CNAME)

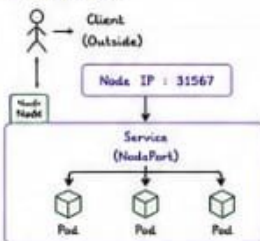
③ SERVICE TYPES EXPLAINED

A. CLUSTERIP (Default)



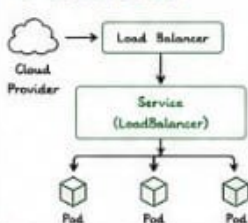
- Exposes the Service on a cluster-internal IP.
- Only reachable from inside the cluster.
- Best for internal microservices.

B. NODEPORT



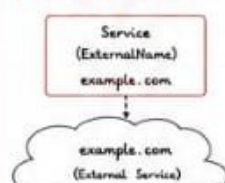
- Exposes the Service on each node's IP at a static port (30000-32767).
- Reachable from outside using <NodeIP>:<NodePort>.
- Good for testing or bare metal environments.

C. LOADBALANCER



- Creates an external Load Balancer in the cloud.
- Exposes a stable external IP.
- Best for production workloads in the cloud.
- Routes traffic to Nodes and then to Pods.

D. EXTERNALNAME



- Maps the Service to an external DNS name.
- No proxying. Returns a CNAME record.
- Used to access external services (e.g. databases, APIs).

④ SERVICE DISCOVERY

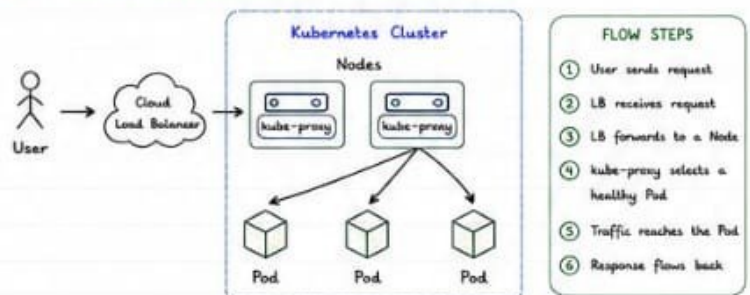
- Kubernetes DNS is powered by CoreDNS.
- Every Service gets a DNS name automatically.
- Format:
`<service-name>.<namespace>.svc.cluster.local`
- Example:
`my-service.default.svc.cluster.local`
- Pods can reach Services using the DNS name instead of IP.

EXAMPLE

Service Name: my-service
Namespace: default
DNS Name:
`my-service.default.svc.cluster.local`

★ Think: DNS + Services = **Loose coupling + Resilience**.

⑤ TRAFFIC FLOW (EXAMPLE: LOADBALANCER)



FLOW STEPS

- ① User sends request
- ② LB receives request
- ③ LB forwards to a Node
- ④ kube-proxy selects a healthy Pod
- ⑤ Traffic reaches the Pod
- ⑥ Response flows back



Think: Services + Endpoints + kube-proxy
make traffic flow reliably to the right Pods.

⑥ PRODUCTION BEST PRACTICES

- ✓ Prefer ClusterIP for internal services.
- ✓ Use LoadBalancer for production external access.
- ✓ Restrict NodePort to trusted environments.
- ✓ Use meaningful names for Services.
- ✓ Group related Services in a namespace.
- ✓ Monitor Service endpoints and health.
- ✓ Use NetworkPolicies to control traffic.

QUICK COMMANDS

```
# Get all services
kubectl get svc

# Describe a service
kubectl describe svc <name>

# Expose deployment as service
kubectl expose deploy myapp \
  --type=ClusterIP --port=80

# Get endpoints of a service
kubectl get endpoints <name>
```

★ KEY TAKEAWAY



Services provide **stable** access to your applications. Choose the **right type**, enable **discovery**, and let Kubernetes handle the network magic!

REMEMBER



- Pods change.
- Services stay.
- DNS finds them.
- Traffic flows.
- Your app scales.

Kubernetes makes it simple!

“ Services connect the world to your workloads – **reliably, scalably, and intelligently**. ”



12. CONFIGMAPS

“ Store configuration separately from your containers. ”

① WHAT IS A CONFIGMAP?

- ConfigMaps store non-sensitive configuration data in key-value pairs.
- They decouple configuration from image, making apps more portable and dynamic.
- Used by Pods as environment variables or mounted files.

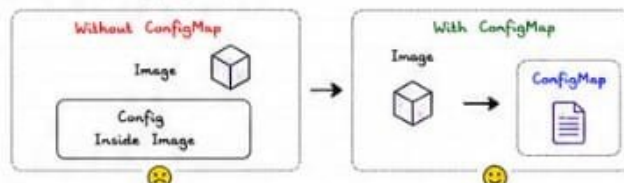
EXAMPLE CONFIGMAP

```
yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_NAME: "VeriQTAApp"
  LOG_LEVEL: "info"
  FEATURE_X: "true"
```

☆ Think: ConfigMaps = Externalized configuration.

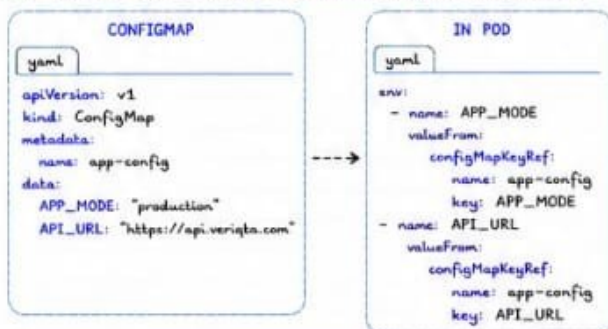
② CONFIGURATION MANAGEMENT

- Keep configuration separate from code.
- Easy to update without rebuilding images.
- Promote consistency across environments (dev, staging, prod).
- Supports 12-Factor App principle.



💡 Think: Separate config from code for flexibility and portability.

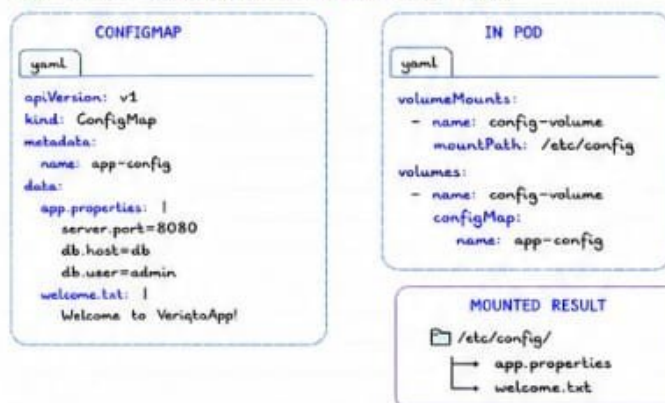
③ USING CONFIGMAPS AS ENVIRONMENT VARIABLES



☆ Use envFrom to load all keys as environment variables.

```
envFrom:
- configMapRef:
  name: app-config
```

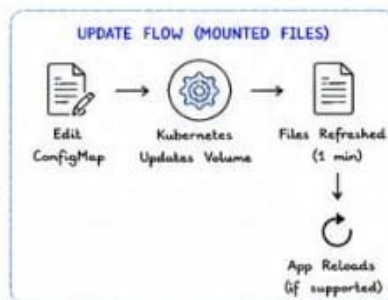
④ USING CONFIGMAPS AS MOUNTED FILES



💡 Files are created inside the container from ConfigMap keys.

⑤ UPDATE BEHAVIOR

- ConfigMap changes do NOT automatically restart Pods.
- For environment variables:
 - Pod must be restarted.
- For mounted files:
 - Updates are reflected automatically (approx. within 1 minute).
 - Applications must reload files to see changes.



☆ Best practice: Use checksums & rollout restart for critical configs.

⑦ BEST PRACTICES

- ✓ Use meaningful names and keys.
- ✓ Group related configs in one ConfigMap.
- ✓ Keep data non-sensitive (use Secrets for sensitive data).
- ✓ Use immutable ConfigMaps for critical configs.
- ✓ Use checksums annotation to trigger rollouts.
- ✓ Version your ConfigMaps when needed.
- ✓ Document all configuration keys.
- ✓ Limit permissions with RBAC.

IMMUTABLE CONFIGMAP (K8S 1.19+)

```
yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config-v1
immutable: true
data:
  key: value
```

> Trigger rollout after ConfigMap update (env vars):
kubectl rollout restart deployment <deployment-name>

★ KEY TAKEAWAY

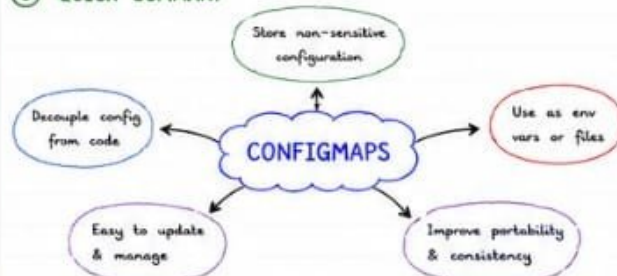
ConfigMaps keep your applications configurable, flexible, and environment-agnostic.



> COMMON COMMANDS

```
# Create ConfigMap from literal
kubectl create configmap app-config --from-literal=key=value
# Create from file
kubectl create configmap app-config --from-file=app.properties
# Get / Describe
kubectl get configmap
kubectl describe configmap app-config
```

⑧ QUICK SUMMARY



REMEMBER

- ✓ Not for secrets.
- ✓ Pods don't auto-restart on env var changes.
- ✓ Mounted files update after ~1 minute.
- ✓ Plan rollouts for critical configuration changes.





13. SECRETS

“ Secrets store sensitive data such as passwords, tokens, and keys. ”

① WHAT IS A SECRET?

- Secrets store sensitive information such as passwords, tokens, SSH keys, or certificates.
- They are stored as key-value pairs.
- Kubernetes does not encrypt Secrets by default (etcd encryption optional).
- Use Secrets instead of ConfigMaps for sensitive data.

EXAMPLE SECRET

```
yaml
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
type: Opaque
data:
  username: YWRtaW4=
  password: UGFzZ3cwcmQ=
# Values are Base64 encoded
```

☆ Think: Secrets keep secrets.
Don't expose them in ConfigMaps or code!

② SECRET TYPES

TYPE	DESCRIPTION	USE CASE
Opaque	Generic key-value secret.	Passwords, tokens, API keys
kubernetes.io/dockerconfigjson	Docker registry credentials.	Pull private images from registries
kubernetes.io/tls	TLS data (cert & private key).	Ingress TLS, mTLS, web servers
kubernetes.io/ssh-auth	SSH private key.	SSH access to pods/nodes
bootstrap.kubernetes.io/token	Cluster bootstrap token.	Node joining



Think: Use the right type for the right secret.

③ BASE64 ENCODING

- Secret data is Base64 encoded (not encrypted).
- Anyone with access can decode it.
- It's just a transport encoding.

ENCODE

```
echo -n 'mypassword' |
base64
```

Output:
bXl0eXNzd29yZA==

DECODE

```
echo 'bXl0eXNzd29yZA==' |
base64 --decode
```

Output:
mypassword

☆ Think: Base64 ≠ Security. It's reversible!

④ USING SECRETS AS ENVIRONMENT VARIABLES

IN POD YAML

```
env:
- name: DB_USER
  valueFrom:
    secretKeyRef:
      name: db-credentials
      key: username
- name: DB_PASS
  valueFrom:
    secretKeyRef:
      name: db-credentials
      key: password
```



Secrets become env vars inside the container.

⑤ USING SECRETS AS MOUNTED FILES

IN POD YAML

```
volumeMounts:
- name: secret-volume
  mountPath: /etc/secret
  readOnly: true
volumes:
- name: secret-volume
  secret:
    secretName: db-credentials
```

FILES INSIDE CONTAINER

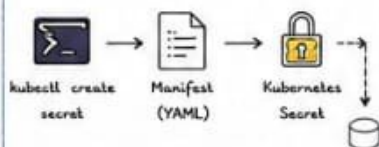
```
/etc/secret/
├── username
└── password
```



Secrets are mounted as files.

⑥ SECRET MANAGEMENT

CREATING A SECRET



WAYS TO CREATE SECRETS

- Imperative: `kubectl create secret ...`
- Declarative: YAML manifest
- From files
- From literals
- From docker config
- From TLS files

EXAMPLE: CREATE FROM LITERAL

```
kubectl create secret generic api-key \
--from-literal=key=abcd1234
```

EXAMPLE: CREATE FROM FILE

```
kubectl create secret generic tls-secret \
--from-file=tls.crt=cert.crt \
--from-file=tls.key=key.key
```

IMPORTANT

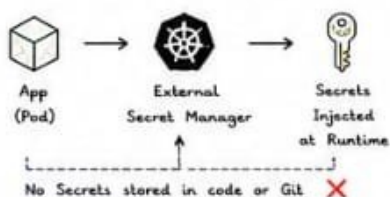


Secrets are stored in etcd.
Enable encryption at rest for etcd in production!
(EncryptionConfiguration)

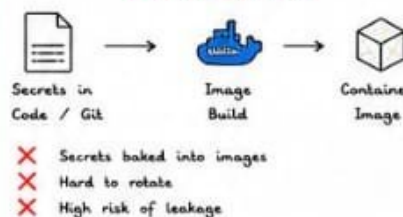
⑦ SECURITY BEST PRACTICES

- ✓ Enable etcd encryption at rest.
- ✓ Restrict access with RBAC.
- ✓ Limit who can create/update Secrets.
- ✓ Avoid storing Secrets in Git repositories.
- ✓ Use external secret managers (e.g., HashiCorp Vault, AWS Secrets Manager).
- ✓ Rotate Secrets regularly.
- ✓ Use least privilege for ServiceAccounts.
- ✓ Audit access to Secrets.

BETTER PRACTICE ARCHITECTURE



NOT RECOMMENDED



★ KEY TAKEAWAY

Secrets protect sensitive data. Use the right type, manage access carefully, and never store Secrets in code or images.



QUICK COMMANDS

```
# Create secret from literal
kubectl create secret generic my-secret --from-literal=password=pass123
# Create secret from file
kubectl create secret generic my-secret --from-file=key=./key.txt
# Get secret (decoded)
kubectl get secret my-secret -o yaml
```

REMEMBER

- ✓ Base64 is not encryption.
- ✓ Enable encryption at rest.
- ✓ Use RBAC to control access.
- ✓ Prefer external secret managers.
- ✓ Keep Secrets out of source code.



“ Protect your secrets like your clusters – with layers, not luck. ”

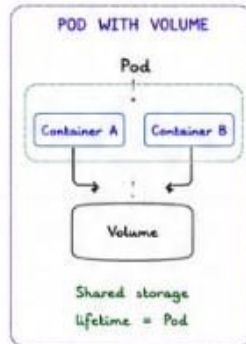


14. VOLUMES

“ Volumes let your containers store, share, and persist data. ”

① WHAT IS A VOLUME?

- Volumes provide storage that is accessible to containers in a Pod.
- They persist data beyond the lifetime of a container.
- Useful for sharing data between containers.
- Kubernetes supports many volume types.



★ Think: Volumes = Data for your containers.

② VOLUME TYPES OVERVIEW

TYPE	DESCRIPTION	LIFETIME	USE CASE
emptyDir	Ephemeral storage created when Pod is assigned to a node	Pod lifetime	Scratch space, caching, temp files
hostPath	Mounts a file or directory from the node's filesystem	Pod lifetime (but data on node persists)	Access node files, logs, local tools
configMap	Mounts ConfigMap as files	Pod lifetime	App configuration files
secret	Mounts Secret as files	Pod lifetime	Passwords, tokens, certs, keys
Persistent Volume (PV)	Cluster storage provisioned outside Pod	Independent of Pod	Databases, stateful apps, persistence



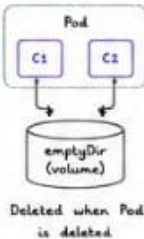
Think: Choose the right volume for the right need.

③ EMPTYDIR (EPHEMERAL)

USE CASES

- Scratch space
- Caching
- Temporary files
- Inter-container communication

DIAGRAM



EXAMPLE

```

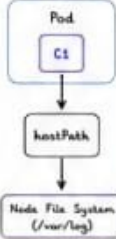
volumes:
- name: temp-volume
  emptyDir: {}
volumeMounts:
- name: temp-volume
  mountPath: /tmp
  
```

④ HOSTPATH

USE CASES

- Access node files
- Logs access
- Running local tools
- Testing / dev workloads

DIAGRAM



EXAMPLE

```

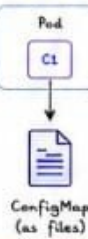
volumes:
- name: host-volume
  hostPath:
    path: /var/log
    type: Directory
volumeMounts:
- name: host-volume
  mountPath: /host/logs
  
```

⑤ CONFIGMAP VOLUME

USE CASES

- Application config
- Feature flags
- Nginx config
- Environment specific files

DIAGRAM



EXAMPLE

```

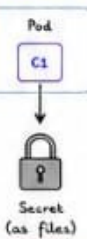
volumes:
- name: config-volume
  configMap:
    name: app-config
volumeMounts:
- name: config-volume
  mountPath: /etc/config
  
```

⑥ SECRET VOLUME

USE CASES

- Passwords
- API tokens
- Certificates
- SSH keys

DIAGRAM



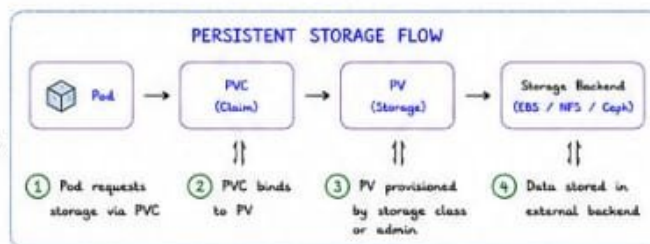
EXAMPLE

```

volumes:
- name: secret-volume
  secret:
    secretName: db-credentials
volumeMounts:
- name: secret-volume
  mountPath: /etc/secret
  
```

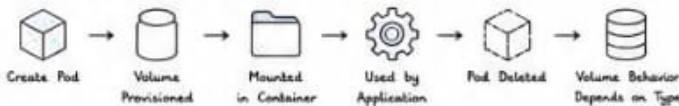
⑦ SHARED STORAGE OPTIONS

- When data must survive Pod restarts or be used across multiple Pods/nodes, use persistent storage.
- Common options:
 - PersistentVolume (PV) + PersistentVolumeClaim (PVC)
 - NFS
 - EBS (AWS)
 - Azure Disk
 - Ceph / GlusterFS



Think: Use PV/PVC for data that must survive Pod death.

⑧ VOLUME LIFECYCLE



TYPE	AFTER POD DELETION
emptyDir	Deleted automatically
hostPath	Data remains on node (not managed by K8s)
ConfigMap / Secret	Files removed (ConfigMap/Secret still exist in K8s)
PersistentVolumeClaim	Data remains (depends on reclaim policy: Retain/Delete)



Think: Volume lifecycle varies by type. Know what stays and what goes!

⑨ PRODUCTION BEST PRACTICES

- ✓ Use emptyDir for temporary data only.
- ✓ Avoid hostPath in production (use only when necessary).
- ✓ Store configuration in ConfigMaps, not in container images.
- ✓ Store sensitive data in Secrets, not in ConfigMaps.
- ✓ Use PV/PVC for data that must persist.
- ✓ Use least-privilege access to Secrets (RBAC).
- ✓ Enable encryption at rest for Secrets (etcd encryption).
- ✓ Rotate Secrets regularly.
- ✓ Monitor disk usage on nodes and volumes.
- ✓ Document and label all volumes clearly.

BEST PRACTICE
FORMULA

Right Data
+
Right Volume
+
Right Security
=
Reliable Applications

★ KEY TAKEAWAY

Volumes give your Pods the ability to store, share, and persist data. Choose the right type and manage them securely.



🔧 QUICK COMMANDS

```

# List volumes in a Pod
kubectl describe pod <pod-name> | grep -AS Volumes

# View PVCs
kubectl get pvc

# View PVs
kubectl get pv
  
```

```

# Describe PV
kubectl describe pv <pv-name>

# Describe StorageClass
kubectl get storageclass

# Get Secrets (names only)
kubectl get secrets
  
```

REMEMBER

- ✓ Don't store secrets in images.
- ✓ Choose volumes intentionally.
- ✓ Persist what you must.
- ✓ Secure everything.
- ✓ Monitor always.





15. PERSISTENT VOLUMES

“ Persistent storage for data that must outlive Pods. ”

① PERSISTENT VOLUME (PV)

- A Persistent Volume is a piece of storage in the cluster provisioned by an admin.
- It is a cluster resource independent of any Pod.
- PVs have a lifecycle independent of Pods.
- They store data reliably using storage from cloud providers or on-prem systems.

PV YAML EXAMPLE

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-01
spec:
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: manual
  hostPath:
    path: /data/pv-01
```

★ Think: PV = The storage itself in the cluster.

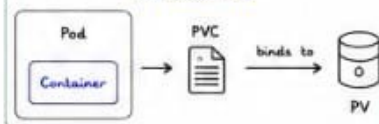
② PERSISTENT VOLUME CLAIM (PVC)

- A PVC is a request for storage by a user.
- It consumes storage from a PV.
- Pods use PVCs as volumes.
- PVCs abstract storage details from application developers.
- They live in a Namespace.

PVC YAML EXAMPLE

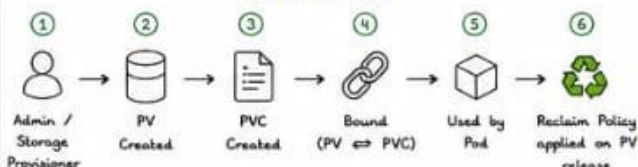
```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  storageClassName: standard
```

POD USING PVC

💡 Think: PVC = The request for storage.
PV = The actual storage.

③ STORAGE LIFECYCLE

LIFECYCLE FLOW



PV RECLAIM POLICIES

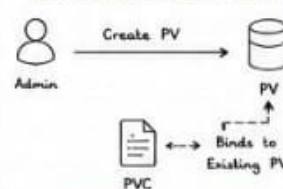
POLICY	DESCRIPTION
Retain	PV is not deleted. Data is kept. Manual cleanup.
Delete	PV is deleted by Kubernetes. (Cloud disk removed)
Recycle (Deprecated)	Data is wiped (rm -rf /thevolume/*) then made available.

★ Think: PV lifecycle is independent of the Pod.
Data stays even if Pod is deleted.

④ STATIC PROVISIONING

- Admin manually creates PVs in advance.
- PVCs bind to existing PVs based on capacity, accessModes, and storageClass.
- Best for local storage or special requirements.

STATIC PROVISIONING FLOW



USE CASES

- ✓ On-prem storage
- ✓ Pre-provisioned disks
- ✓ Compliance / Data residency
- ✓ Special performance storage

PROS / CONS

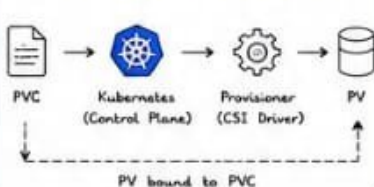
- + Full control
- + Predictable
- Manual effort
- Not elastic

💡 Think: You create storage first, then claim it.

⑤ DYNAMIC PROVISIONING OVERVIEW

- StorageClass defines how storage should be provisioned (e.g., AWS EBS, GCE PD, NFS).
- When a PVC is created, Kubernetes automatically creates a PV using the StorageClass.
- Ideal for cloud environments and scale.

DYNAMIC PROVISIONING FLOW



STORAGECLASS EXAMPLE

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gp3
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
```

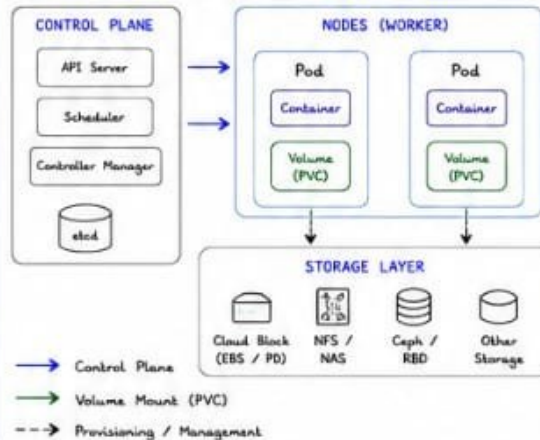
USE CASES

- ✓ Cloud environments
- ✓ Auto-scaling applications
- ✓ Dev/Test environments
- ✓ Multi-tenant clusters

★ Think: You request storage, Kubernetes creates it for you.

⑥ STORAGE ARCHITECTURE

KUBERNETES CLUSTER



★ KEY TAKEAWAY

- ✓ PV is the storage in the cluster.
- ✓ PVC is the storage request.
- ✓ PV and PVC work together.
- ✓ Use StorageClass for dynamic provisioning.
- ✓ Choose the right storage for the right workload.

🔧 QUICK COMMANDS

```
# List PVs
kubectl get pv
# List PVCs
kubectl get pvc
# Describe PV
kubectl describe pv <pv-name>
# Describe PVC
kubectl describe pvc <pvc-name>
# Delete PVC
kubectl delete pvc <pvc-name>
```

REMEMBER

- ✓ Use appropriate accessModes.
- ✓ Set correct reclaimPolicy.
- ✓ Monitor storage usage.
- ✓ Back up important data.
- ✓ Test restore procedures.



“ Good storage design today prevents data loss tomorrow. ”



16. RESOURCE REQUESTS & LIMITS

“Manage resources wisely to get stability, performance, and fairness.”

① WHY REQUESTS & LIMITS?

- Kubernetes schedules Pods based on requests, not limits.
- Prevents one container from consuming all resources.
- Improves cluster stability and application performance.
- Enables fair sharing of resources.
- Required for features like QoS classes and HPA.

★ Think:

Requests = What you need to run.

Limits = The maximum you can use.

② CPU & MEMORY CONCEPTS

TYPE	DESCRIPTION	UNIT	EXAMPLE
CPU Request	Minimum CPU guaranteed to the container	cores or millicores (1 core = 1000m)	500m, 1, 2
Memory Request	Minimum memory guaranteed to the container	Bytes (Mi, Gi)	128Mi, 256Mi, 1Gi
CPU Limit	Maximum CPU allowed for the container	cores or millicores	1, 2, 2000m
Memory Limit	Maximum memory allowed for the container	Bytes (Mi, Gi)	512Mi, 2Gi



Think: Set requests first, then set limits.

③ EXAMPLE: RESOURCE SPEC IN POD

POD YAML EXAMPLE

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx:latest
    resources:
      requests:
        cpu: "500m"
        memory: "256Mi"
      limits:
        cpu: "1000m"
        memory: "512Mi"
```

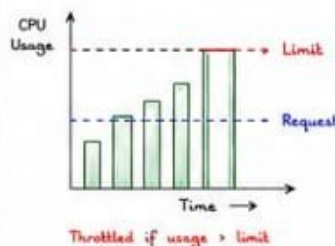
Minimum guaranteed resources

Maximum allowed resources

④ RESOURCE BEHAVIOR

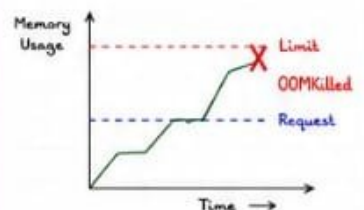
CPU

- CPU is a compressible resource.
- Container can burst above request up to the limit.
- If all nodes are busy, may get throttled at limit.



MEMORY

- Memory is not compressible.
- If a container exceeds its memory limit, it will be killed (OOMKilled).
- Requests help the scheduler place Pods.



⑤ QoS CLASSES

CLASS	CONDITION	DESCRIPTION	EVICION PRIORITY
Guaranteed	requests = limits for CPU and Memory (and all containers)	Best performance and most stable.	Last (lowest priority)
Burstable	requests < limits or one resource not set	Can burst above requests up to limits.	Middle
BestEffort	No requests and no limits	No guarantees. Least stable.	First (highest priority)

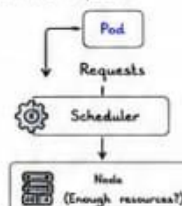


Think: Guaranteed > Burstable > BestEffort

⑥ HOW KUBERNETES USES REQUESTS

SCHEDULING

Kubernetes scheduler makes sure the node has enough available resources to satisfy the Pod's requests.



QUALITY OF SERVICE & EVICTION

During resource pressure, kubelet evicts Pods based on QoS class.



⑦ RESOURCE UNITS REFERENCE

CPU UNITS

- 1 core = 1000 millicores (m)
- 250m = 0.25 core
- 500m = 0.5 core
- 1000m = 1 core
- 2000m = 2 cores



MEMORY UNITS

- 1Ki = 1024
- 1Mi = 1024 Ki
- 1Gi = 1024 Mi
- 1Ti = 1024 Gi



★ Think: Always use units (m for CPU, Mi/Gi for memory).



KEY TAKEAWAY

Requests schedule.
Limits protect.
QoS classes decide survival.



QUICK COMMANDS

```
# Describe Pod (check resources)
kubectl describe pod <pod-name>
# Top Pods (resource usage)
kubectl top pod
# Top Nodes (resource usage)
kubectl top nodes
```

⑧ RESOURCE MANAGEMENT BEST PRACTICES

- ✓ Always set requests and limits.
- ✓ Set requests close to actual usage.
- ✓ Avoid very high limits.
- ✓ Use vertical pod autoscaler for right-sizing.
- ✓ Monitor usage with metrics (Prometheus, Grafana).
- ✓ Test under load and adjust.
- ✓ Use LimitRanges and ResourceQuotas.

RIGHT-SIZE FORMULA

Observe usage
+
Add buffer (20-30%)
=
Set request
Set limit (2-3x request)

REMEMBER

- ✓ Set requests = what you need.
- ✓ Set limits = what you allow.
- ✓ Guaranteed Pods are most stable.
- ✓ Monitor and adjust regularly.



“Good resource management today prevents outages tomorrow.”



17. KUBERNETES YAML ANATOMY

"YAML is how you describe what you want. Kubernetes makes it real."

① WHAT IS KUBERNETES YAML?

- YAML (YAML Ain't Markup Language) is the declarative language used to define Kubernetes objects.
- You describe the desired state, Kubernetes handles the actual state.
- YAML files are applied using `kubectl apply -f file.yaml`
- It is human-readable, structured, and indentation-sensitive.

☆ Think:

YAML = Your instructions

Kubernetes = The executor

② YAML ANATOMY - THE 5 MAIN SECTIONS

① apiVersion	Defines the API group and version of the Kubernetes object.	Example: <code>apps/v1</code>
② kind	Specifies the type of Kubernetes object (e.g., Pod, Deployment, Service).	Example: <code>Deployment</code>
③ metadata	Provides data to uniquely identify the object (name, namespace, labels, annotations).	Example: <code>name: my-app</code>
④ spec	Defines the desired state of the object. This is the most important section.	Example: <code>replicas: 3</code>
⑤ status	Shows the current state of the object. (Managed by Kubernetes, not by you.)	Example: <code>readyReplicas: 3</code>



Think: You define **apiVersion**, **kind**, **metadata**, **spec**. Kubernetes fills **status**.

③ EXAMPLE: A SIMPLE DEPLOYMENT YAML

```

1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: my-app
5   namespace: default
6   labels:
7     app: my-app
8 spec:
9   replicas: 3
10  selector:
11    matchLabels:
12      app: my-app
13  template:
14    metadata:
15      labels:
16        app: my-app
17    spec:
18      containers:
19        - name: nginx
20          image: nginx:latest
  
```

Annotations in the image:

- `apiVersion` and `kind` are grouped together.
- `metadata` is highlighted in green.
- `spec` is highlighted in blue and labeled "(desired state)".
- `status` is indicated as "(status will be added by K8s)".

④ DETAILED EXPLANATION OF EACH SECTION

① apiVersion	- Determines which API group and version is used - Different object types have different API groups - Example: <code>apps/v1</code>, <code>v1</code>, <code>batch/v1</code>	Tells Kubernetes which API to use.
② kind	- The type of object you're creating - Examples: Pod, Deployment, Service, ConfigMap - Must be a valid Kubernetes object	Tells Kubernetes what object to create.
③ metadata	<code>name</code>: Unique name of the object <code>namespace</code>: Where the object lives <code>labels</code>: Key-value pairs for grouping <code>annotations</code>: Extra info (optional)	Helps identify and organize objects.
④ spec	- The desired state configuration - What you want Kubernetes to do - Varies by object type	Defines the desired state.
⑤ status	- The current state reported by Kubernetes - Read-only for users - Helps you see what is actually happening	Shows the actual state.

⑤ READING ANY KUBERNETES YAML

- Check **apiVersion** → Which API group/version?
- Check **kind** → What type of object is this?
- Check **metadata** → What is its name, namespace, labels?
- Check **spec** → What is the desired state?
- Ignore **status** → Kubernetes manages it.

☆ Think: Focus on 1-4 when creating or editing YAML. Status is for information only.

⑥ COMMON INDENTATION RULES

- Use 2 spaces per indentation level (never tabs).
- Indentation defines structure.
- A wrong indent = Invalid YAML!

CORRECT ✓

```
spec:
  replicas: 3
  template:
    spec:
      containers:
        - name: nginx
```

INCORRECT ✗

```
spec:
  replicas: 3
  template:
    spec:
      containers:
        - name: nginx
```

⑦ FULL YAML WITH STATUS (AFTER APPLY)

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
  namespace: default
  labels:
    app: my-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    ...
status:
  observedGeneration: 2
  replicas: 3
  readyReplicas: 3
  availableReplicas: 3
  conditions:
    - type: Available
      status: "True"
  
```

Annotations in the image:

- Sections 1-4 (`apiVersion` to `template`) are grouped and labeled "You define these parts (1-4)".
- The `status` section is labeled "Kubernetes updates this (status)".

TIP

You declare the desired state.

Kubernetes continuously reconciles it to match the actual state.



★ KEY TAKEAWAY

Understand the 5 main sections.
Focus on **spec** when writing YAML.
Let Kubernetes handle the rest.



📄 QUICK COMMANDS

```

kubectl apply -f file.yaml # Create or update
kubectl get all           # See resources
kubectl describe <resource> <name> # Details
kubectl edit <resource> <name>    # Edit live
kubectl diff -f file.yaml  # Preview changes
  
```

REMEMBER

- ✓ **apiVersion** - Which API?
- ✓ **kind** - What object?
- ✓ **metadata** - Identity
- ✓ **spec** - Desired state
- ✓ **status** - Actual state (read-only)



"Good YAML is simple, clear, and correct. Great YAML is maintainable."

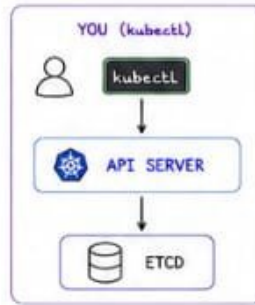


18. KUBECTL ESSENTIALS

"Kubectl is your remote control for the Kubernetes cluster."

① WHAT IS KUBECTL?

- kubectl is the CLI tool to interact with Kubernetes clusters.
- It lets you create, inspect, update and manage K8s resources.
- Communicates with the API Server.
- Works anywhere you have network access & kubeconfig.



☆ Think: kubectl = Power in your terminal.

② KUBECTL COMMANDS OVERVIEW

COMMAND	PURPOSE	EXAMPLE
get	List resources	kubectl get pods
describe	Show details of a resource	kubectl describe pod nginx
logs	View logs from a container	kubectl logs -f pod/nginx
exec	Execute command in a container	kubectl exec -it pod/nginx -- bash
apply	Create / Update resources from YAML	kubectl apply -f app.yaml
delete	Delete resources	kubectl delete pod nginx



Think: Get → Inspect → Debug → Create → Clean up
(The kubectl lifecycle)

③ KUBECTL COMMANDS CHEAT SHEET

CATEGORY	COMMAND SYNTAX	EXAMPLE	DESCRIPTION
GET (List resources)	kubectl get <resource> [options]	- kubectl get pods - kubectl get svc - kubectl get deployments -n dev - kubectl get all - kubectl get pods -o wide	- List resources. -n <ns> : Specify namespace -o wide : More details -o yaml/json : Output format
DESCRIBE (Details)	kubectl describe <resource> <name> [options]	- kubectl describe pod nginx - kubectl describe svc my-service - kubectl describe node worker-1	- Show detailed information - Includes events, status, conditions
LOGS (Logs)	kubectl logs <pod-name> [options]	- kubectl logs nginx - kubectl logs -f nginx - kubectl logs -f nginx -c app	-f : Follow logs -c <container> : For multi-container pods --since=10m : Last 10 minutes
EXEC (Execute)	kubectl exec [options] <pod-name> -- <command>	- kubectl exec -it nginx -- bash - kubectl exec -it nginx -- sh - kubectl exec nginx -- ls /app	-it : Interactive terminal - Run commands inside container
APPLY (Create / Update)	kubectl apply -f <file.yaml> [-f <dir/>]	- kubectl apply -f app.yaml - kubectl apply -f ./manifests/	- Create or update resources - Idempotent (safe to run multiple times)
DELETE (Delete)	kubectl delete <resource> <name> [options]	- kubectl delete pod nginx - kubectl delete -f app.yaml - kubectl delete deployment my-app	- Delete resources --all : Delete all of a type -n <ns> : Specify namespace

④ COMMON RESOURCE SHORTCUTS

SHORTCUT	RESOURCE	SHORTCUT	RESOURCE
po	pods	ns	namespaces
svc	services	no	nodes
deploy	deployments	pv	persistentvolumes
rs	replicasets	pvc	persistentvolumeclaims
cm	configmaps	ing	ingresses
sec	secrets	ep	endpoints

☆ Think: kubectl get po == kubectl get pods

⑤ EXAMPLES IN ACTION

- List all pods
\$ kubectl get pods # Show running pods
- Describe a pod
\$ kubectl describe pod nginx # Show details & events
- View logs (Follow)
\$ kubectl logs -f nginx # Stream logs
- Exec into a pod
\$ kubectl exec -it nginx -- bash # Open shell
- Apply configuration
\$ kubectl apply -f app.yaml # Create/Update from YAML
- Delete a pod
\$ kubectl delete pod nginx # Delete resource

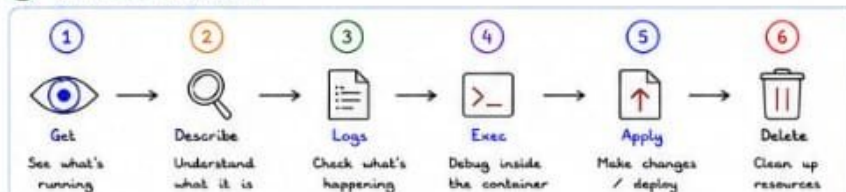
⑥ FLAGS YOU WILL USE OFTEN

FLAG	MEANING
-n, --namespace	Specify namespace
-o wide	Wide output
-o yaml	YAML output
-o json	JSON output
-l key=value	Filter by label
--all	Select all resources
-f	Filename or directory
-it	Interactive terminal
--grace-period=0 --force	Force delete



Think: Flags make kubectl powerful. Use them!

⑦ PRACTICE WORKFLOW



☆ Think: Observe → Understand → Debug → Change → Maintain → Clean



KEY TAKEAWAY

kubectl is your daily driver. Know a few commands, and you control the cluster.



QUICK REFERENCE

get : See
describe : Understand
logs : Check
exec : Debug
apply : Change
delete : Clean up



GET BETTER

Practice daily
Use --help
Read the docs
Build muscle memory

REMEMBER

- Start simple.
- Use flags.
- Read outputs.
- Keep learning.



"The more you kubectl, the more you Kube-it."

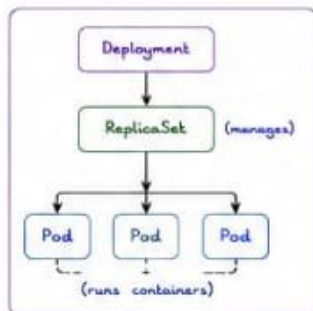


19. KUBERNETES OBJECT RELATIONSHIPS

“ Objects work together to run your applications. ”

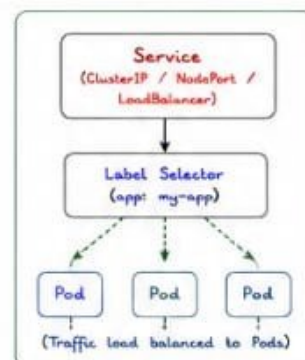
① DEPLOYMENT → REPLICASET → POD

- Deployment is a high-level controller.
- It creates and manages ReplicaSets.
- ReplicaSet ensures the desired number of Pods are running.
- Pods are the actual containers.



② SERVICE → POD

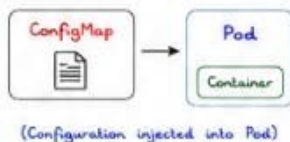
- Service provides a stable network endpoint.
- It load balances traffic across matching Pods.
- Pods are discovered using labels and selectors.



★ Think: Controllers manage Pods. Services expose Pods. ConfigMaps, Secrets and Volumes provide data to Pods.

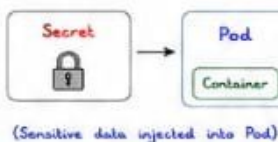
③ CONFIGMAP → POD

- ConfigMap stores non-sensitive configuration.
- It can be used as environment variables or mounted as files.
- Pods consume configuration at runtime.



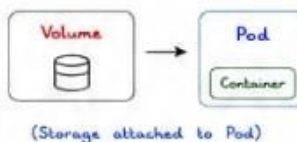
④ SECRET → POD

- Secret stores sensitive data (passwords, tokens, keys).
- Used as environment variables or mounted as files.
- Pods access secrets safely.



⑤ VOLUME → POD

- Volume provides storage to Pods.
- Data can be temporary or persistent.
- Volumes can come from many sources (emptyDir, PVC, etc).

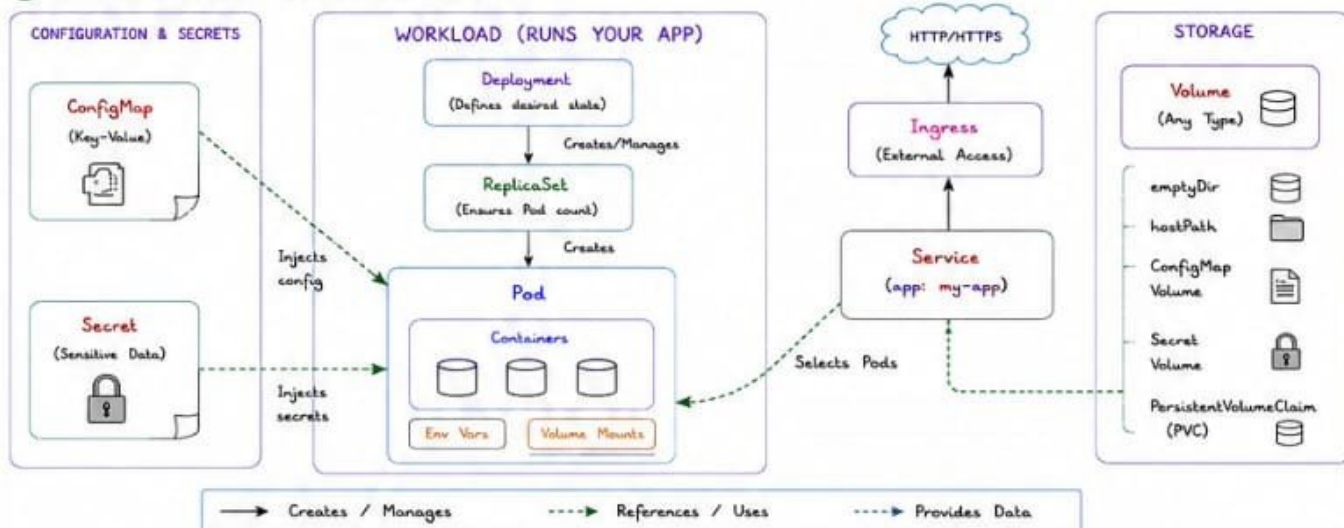


⑥ OTHER COMMON RELATIONSHIPS

- Ingress → Service → Pod
- StatefulSet → Pod
- DaemonSet → Pod
- Job → Pod
- CronJob → Job → Pod
- HPA → Deployment / ReplicaSet → Pod

💡 Think: Everything connects to Pods!

⑦ COMPLETE OBJECT RELATIONSHIP DIAGRAM



⑧ RELATIONSHIP SUMMARY TABLE

FROM	TO	RELATIONSHIP	PURPOSE
Deployment	ReplicaSet	Creates / Manages	Manages ReplicaSet and rollout
ReplicaSet	Pod	Creates	Ensures desired number of Pods
Service	Pod	Selects / Routes	Routes traffic to matching Pods
ConfigMap	Pod	Injects	Provides non-sensitive configuration
Secret	Pod	Injects	Provides sensitive data
Volume	Pod	Attaches	Provides storage to containers
Ingress	Service	Routes	External access to Services

⑨ KEY TAKEAWAYS

- ✓ Controllers create and manage Pods.
- ✓ Services provide stable networking to Pods.
- ✓ ConfigMaps and Secrets provide data to Pods.
- ✓ Volumes provide storage to Pods.
- ✓ Labels and Selectors are the glue!



★ Think: Know the relationships, understand the architecture, troubleshoot with confidence.

BEST PRACTICES

- ✓ Use labels consistently
- ✓ Keep config in ConfigMaps
- ✓ Store secrets in Secrets
- ✓ Use appropriate volume types
- ✓ Follow least privilege access



QUICK CHECK COMMANDS

```
# See all Pods and their owners
kubectl get pods -o wide
# See relationships (ownerReferences)
kubectl describe pod <pod-name>
# See services and endpoints
kubectl get svc,ep
```

```
# See configmaps and secrets
kubectl get configmap,secret
# See volumes in a pod
kubectl describe pod <pod-name> | grep -A 10 'Volumes:'
```

REMEMBER

- ✓ Controllers manage Pods
- ✓ Services expose Pods
- ✓ Data comes from ConfigMaps & Secrets
- ✓ Storage comes from Volumes



“ Understand the relationships. Master Kubernetes. ”

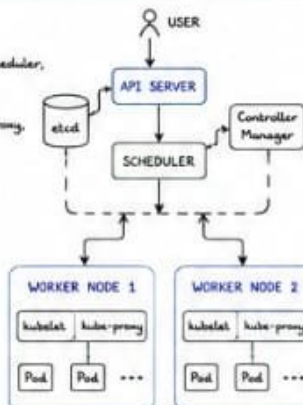
20. KUBERNETES FUNDAMENTALS
MASTER CHECKLIST

“ Foundation today. Production ready tomorrow. ”

★ Goal: Master the core building blocks of Kubernetes and be confident to build, run and troubleshoot workloads.

1 ARCHITECTURE REVIEW

- ☐ Understand Kubernetes architecture.
- ☐ Control Plane components (API Server, Scheduler, Controller Manager, etcd).
- ☐ Worker Node components (kubelet, kube-proxy, container runtime).
- ☐ How requests flow from user to Pod.
- ☐ Understand etcd as the source of truth.
- ☐ Know how the scheduler works.
- ☐ Understand basic HA and scaling.
- ☐ Draw the architecture from memory.



2 OBJECT REVIEW

- ☐ Understand Pods (smallest deployable unit).
- ☐ Understand Deployments and ReplicaSets.
- ☐ Know Services and their types.
- ☐ Understand ConfigMaps and Secrets.
- ☐ Understand Volumes and Persistent Volumes.
- ☐ Know how Ingress works.
- ☐ Understand resource requests & limits.
- ☐ Know object relationships.
- ☐ Can you explain how objects work together?



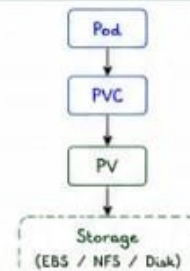
3 NETWORKING BASICS

- ☐ Understand Pod IP and Network.
- ☐ Know ClusterIP, NodePort, LoadBalancer, ExternalName.
- ☐ Understand Service discovery.
- ☐ Understand how traffic flows inside the cluster.
- ☐ Understand Ingress basics.



4 STORAGE BASICS

- ☐ Understand Volumes (emptyDir, hostPath, ConfigMap, Secret).
- ☐ Understand PersistentVolumes (PV).
- ☐ Understand PersistentVolumeClaims (PVC).
- ☐ Know static vs dynamic provisioning.
- ☐ Understand storage lifecycle.
- ☐ Know how storage is used by Pods.



5 YAML REVIEW

- ☐ Understand YAML structure.
- ☐ Know apiVersion, kind, metadata, spec, status.
- ☐ Can you read any Kubernetes YAML?
- ☐ Can you create valid YAML for common objects?
- ☐ Understand indentation rules.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
  labels:
    app: my-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: nginx
          image: nginx:latest
```

6 KUBECTL REVIEW

- ☐ Know basic kubectl syntax.
- ☐ Can you get, describe, logs, exec into resources?
- ☐ Can you apply and delete resources?
- ☐ Know useful flags (-n, -o wide, -f, -i, -all-namespaces).
- ☐ Use kubectl confidently.

KUBECTL COMMAND CHEAT SHEET

kubectl get pods	List pods
kubectl get all	List all resources
kubectl describe pod <name>	Show details
kubectl logs <pod-name>	View logs
kubectl exec -it <pod> -- bash	Open shell
kubectl apply -f <file.yaml>	Create / Update
kubectl delete -f <file.yaml>	Delete
kubectl get pods -n dev	Use namespace
kubectl get pods -o wide	Wide output

7 SELF-ASSESSMENT CHECKLIST

TOPIC	I UNDERSTAND	I CAN EXPLAIN	I CAN APPLY
Architecture	☐	☐	☐
Pods & Controllers	☐	☐	☐
Services & Networking	☐	☐	☐
ConfigMap & Secret	☐	☐	☐
Storage & Volumes	☐	☐	☐
Resource Requests & Limits	☐	☐	☐
YAML Anatomy	☐	☐	☐
kubectl Basics	☐	☐	☐
Object Relationships	☐	☐	☐
Troubleshooting Basics	☐	☐	☐

SCORE GUIDE

24 - 30 🟢 Excellent
You are production-ready!16 - 23 🟡 Good
Keep practicing and building.0 - 15 🔴 Needs Work
Review and practice more.

8 FINAL READINESS ASSESSMENT

Can you do the following without help?

- ☐ Deploy an application using YAML.
- ☐ Expose it using a Service.
- ☐ Access it via browser using NodePort/Ingress.
- ☐ Store configuration using ConfigMap.
- ☐ Store sensitive data using Secret.
- ☐ Persist data using PVC.
- ☐ Check logs and exec into the Pod.
- ☐ Troubleshoot if something breaks.
- ☐ Explain how all objects work together.

You've Got
This!

BEST PRACTICES TO REMEMBER

- ✓ Use meaningful names and labels.
- ✓ Follow least privilege access.
- ✓ Store secrets securely.
- ✓ Set resource requests and limits.
- ✓ Monitor and observe your applications.
- ✓ Automate and version your configs.



COMMANDS TO PRACTICE DAILY

```
kubectl get all
kubectl describe <resource>
kubectl logs <pod>
kubectl exec -it <pod> -- bash
kubectl apply -f <file.yaml>
kubectl delete -f <file.yaml>
kubectl get pods -n <namespace> -o wide
```

REMEMBER

- ✓ Kubernetes is practice + repetition.
- ✓ Start small, build, break, fix, repeat.
- ✓ Understand the WHY, not just the HOW.
- ✓ You don't need to know everything.
- ✓ Keep learning. Keep building. Keep shipping!



“ ★ Fundamentals strong today, Engineer unstoppable tomorrow. ★ ”